

Pin CRT 用户手册

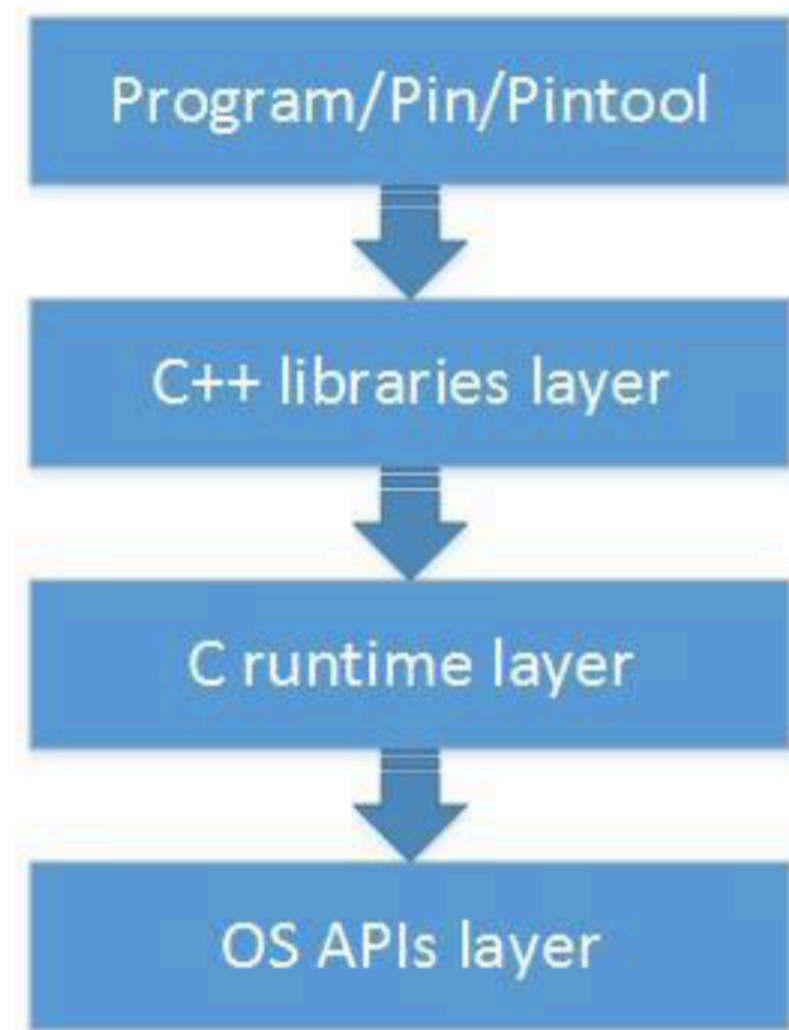
目录

PinCRT 概览	2
使用 PinCRT for Linux 进行开发	4
使用 GCC 进行构建	4
使用 PinCRT for Windows (MSVC Kit) 进行开发	6
使用 Visual Studio 进行构建	6
使用 PinCRT for Windows 进行开发 (Clang-cl 工具包)	10
使用 Visual Studio 进行构建	11
使用 Pin 与 clang-cl 时的问题——已知问题与解决方案	13
从原生 CRT 格式迁移至 PinCRT 格式	16
消除与操作系统相关的函数调用	16
请确保你使用了正确的头文件。	16
请确保将您的 Pin 工具与 PinCRT 库正确关联起来。	17
在 Linux 系统上	17
在 Windows 系统上	18
使用 PinCRT 重新构建任何第三方库	18
迁移过程中可能出现的常见问题	19

PinCRT 概览

PinCRT 架构

PinCRT 由 3 个独立的软件层组成，每一层都依赖于其前面的层：



- 操作系统应用程序接口
 - 实现了一组用于与操作系统进行交互的常用功能。
 - 每个受支持的操作系统都有自己独立的操作系统 API 实现方式。
- C 语言运行时环境
 - 实现了 printf()、strcpy() 等基本 C 语言函数。
 - 负责进程的初始化/终止操作（例如，调用 C++ 的构造函数）。
 - 实现了“编译器运行时功能”——即那些由编译器明确添加的功能（例如，在 32 位架构上对 64 位整数的支持）。
 - 提供了一种与 Pin 加载器进行交互的方式。
 - 基于 Android 的 libc（bionic）实现方式。
- C++ 库/函数库
 - 实现了 C++ 标准库的功能（例如 STL）。
 - 提供有限的 C++ 运行时支持：
 - 不支持异常处理。

- 不支持那些需要运行时类型信息才能执行的操作——例如动态类型转换。
 - 基于 LLVM 12 版本的 libc++ 实现（适用于 Linux 和 Windows 上的 clang-cl 工具链），以及 STLPort 实现（适用于 Windows 上的 MSVC 工具链）。

从技术上讲，PinCRT 到底是什么呢？

- 这是一组头文件，它们基本上可以替代 C 语言的运行时头文件（比如 stdio.h）。
- Pin 工具可以与之链接的一组库（动态库和/或静态库）。

因此，这意味着，要想使用 PinCRT 来创建 Pin 工具，我们必须：

- 使用 PinCRT 的头文件来编译该工具的源代码（需相应地修改包含路径）。
- 将该工具的 objet 文件与 PinCRT 的库文件连接起来（可以是动态连接或静态连接）。

重要提示：

- 与 PinCRT 相关联的 Pin 工具，无法与任何本机系统库一起使用（例如 libc.so、msvcrt*.dll 等）。
- 在启用了 PinCRT 功能的系统中，可以包含几个内置的系统头文件。源文件（相关列表请参见“从原生 CRT 迁移到 PinCRT”一节）。但作为一般原则，在为 PinCRT 进行开发时，不得使用任何原生系统的头文件（应直接使用 PinCRT 所提供的替代文件）。

PinCRT 的功能特点

- 与操作系统无关：在所有平台上，都为 C 语言运行时提供相同的接口和行为。
- 与编译器无关：你可以使用任何编译器来编写代码，然后再将其与 PinCRT 生成的二进制文件进行链接。
- 它拥有自己的线程本地存储机制，不会与 glibc、MS-CRT 等共享线程本地存储空间。这是二进制调试环境所要求的特殊条件。

支持的操作系统

目前，PinCRT 支持以下操作系统：

- Linux
- Windows

使用 PinCRT for Linux 进行开发

使用 GCC 进行构建

编译器参数：

宏指令：

- 适用于所有平台：-D__PIN__=1 -DPIN_CRT=1 -DTARGET_LINUX
- 对于 IA32 架构：-DTARGET_IA32 -DHOST_IA32
- 对于 Intel64 架构：-DTARGET_IA32E -DHOST_IA32E

旗帜：

- 适用于所有编译器：
-funwind-tables -fno-stack-protector -fasynchronous-unwind-tables -fomit-frame-pointer
-fno-strict-aliasing
- 仅适用于 g++：-fno-exceptions -fno-rtti -fPic -faligned-new

包含的路径：

```
-isystem $(PIN_ROOT)/extras/cxx/include \ -isystem $(PIN_ROOT)/extras/crt/include  
\ -isystem $(PIN_ROOT)/extras/crt/include/arch-$(BIONIC_ARCH) \ -isystem  
$(PIN_ROOT)/extras/crt/include/kernel/uapi \ -isystem  
$(PIN_ROOT)/extras/crt/include/kernel/uapi/asm-x86 \ -  
I$(PIN_ROOT)/source/include/pin \ -I$(PIN_ROOT)/source/include/pin/gen \ -  
I$(PIN_ROOT)/extras/components/include \ -I$(PIN_ROOT)/extras/xed-  
(XED_ARCH)/include/xed
```

鉴于：

BIONIC_ARCH 在 IA32 架构下为 “x86” 格式，在 Intel64 架构下则为 “x86_64” 格式。XED_ARCH 在 IA32 架构下为 “ia32” 格式，在 Intel64 架构下则为 “intel64” 格式。PIN_ROOT 则是 Pin 套件的根目录。

链接器标志：

首先，需要移除链接器命令行中所有以 “-l” 表示的依赖库。这些依赖库需要被相应的 PinCRT 版本所替代。

将这些库添加到链接命令行中：

```
-nostdlib -lc-dynamic -lm-dynamic -lc++ -lc++abi -lpindwarf -ldwarf  
L$(PIN_ROOT)/$(TARGET)/runtime/pincrt
```

鉴于：

对于 IA32 架构，TARGET 值为 “ia32”；对于 Intel64 架构，则为 “intel64”。PIN_ROOT 是 Pin 套件的根目录。

使用 PinCRT for Windows (MSVC Kit)进行开发

本节中的说明适用于 Pin Windows MSVC 工具包的使用（即 pinX.XX-buildnum-hash-msvc-windows.zip 文件）。请注意，该工具包提供的 STLPort C++运行时环境仅支持 C++03 标准。

对于基于 LLVM 的 Pin Windows 工具包（pin-X.XX-buildnum-hash-clang-windows.zip），该工具包提供了与 C++11 标准兼容的运行时库。请参阅《在 Windows 上使用 PinCRT 进行构建（Clang-cl 工具包）》的相关说明。

使用 Visual Studio 进行构建

编译器参数：

宏指令：

- 适用于所有平台：/DPIN_CRT=1 /DTARGET_WINDOWS
- 对于 IA32 架构： /DTARGET_IA32 /D__i386__ /DHOST_IA32
- 对于 Intel64 架构：/DTARGET_IA32E /D__LP64__ /DHOST_IA32E
- 用于构建 PinTool 或共享库： /MD
- 如需构建可执行文件或独立运行的工具：请使用/MD 选项。

旗帜：

/GR- /GS- /EHs- /EHa- /FP:严格 /Oi-

包含路径：

```
/I$(PIN_ROOT)/extras/stlport/include /I$(PIN_ROOT)/extras
/I$(PIN_ROOT)/extras/libstdc++/include
/I$(PIN_ROOT)/extras/crt/include /I$(PIN_ROOT)/extras/crt
/I$(PIN_ROOT)/extras/crt/include/arch-$(BIONIC_ARCH)
/I$(PIN_ROOT)/extras/crt/include/kernel/uapi
/I$(PIN_ROOT)/extras/crt/include/kernel/uapi/asm-x86
/I$(PIN_ROOT)/source/include/pin /I$(PIN_ROOT)/source/include/pin/gen
/I$(PIN_ROOT)/extras/components/include
/I$(PIN_ROOT)/extras/xed-$(XED_ARCH)/include/xed
```

鉴于：

BIONIC_ARCH 在 IA32 架构下为“x86”格式，在 Intel64 架构下则为“x86_64”格式。XED_ARCH 在 IA32 架构下为“ia32”格式，在 Intel64 架构下则为“intel64”格式。PIN_ROOT 则是 Pin 套件的根目录。

附加的包含文件：

你应该将此开关添加到编译命令行中，这样在每次编译时都会包含该文件：

```
/FIinclude/msvc_compat.h
```

重要提示：

如果你在某个源文件中引用了 Windows.h：

我们提供了自己的替代文件来替换这个头文件（因为包含 Windows.h 的话，通常也会把所有不需要的 MS-CRT 相关内容一起包含进来）。我们所提供的替代文件实际上是一个“包装器”头文件，它能够确保只有我们需要的内容被声明出来。为此，我们必须知道原始的 Windows.h 文件位于何处——该文件位于 Windows SDK 中。在这种情况下，你需要将这个文件添加到编译参数中。

```
/D_WINDOWS_H_PATH_="$ (ORIGINAL_WINDOWS_H_PATH) "
```

链接器标志：

首先，应从链接器命令行中删除微软的 libc 库文件（例如：libcpmt.lib 或 libcmt.lib）。这些库需要被 PinCRT 版本的库所替代。

将这些导入库添加到链接命令行中：

- 用于构建 PinTool 或共享库：
/NODEFAULTLIB pincrt.lib pinipc.lib kernel32.lib
- 如需构建可执行文件或独立运行的工具：
/NODEFAULTLIB pincrt.lib kernel32.lib

仅支持动态版本的 PinCRT 库。

图书馆搜索路径：

```
/LIBPATH:$(PIN_ROOT)/$(TARGET)/runtime/pincrt  
/LIBPATH:$(PIN_ROOT)/$(TARGET)/lib-ext
```

鉴于：

对于 IA32 架构，TARGET 值为“ia32”；对于 Intel64 架构，则为“intel64”。PIN_ROOT 是 Pin 套件的根目录。

忽略链接器警告：

可以忽略链接器警告#4210 和#4049。只需在链接器命令行中添加相应的参数，即可屏蔽这些警告：

```
/忽略:4210 /忽略:4049
```


需要关联的其他对象/项目

对于每个 libc 来说，都存在一个必须与所有可执行文件/共享对象静态链接的静态部分。具体需要链接哪些目标文件，取决于你是要构建可执行文件还是共享对象。

用于构建共享库或 PinTool:

- 该目标文件必须放在链接命令的首位:

`crtbeginS.o`

如需构建可执行文件或独立运行的工具:

- 该目标文件必须放在链接命令的首位:

`crtbegin.o`

上述目标文件位于以下位置:

`$(PIN_ROOT)/$(TARGET)/runtime/pincrt`

鉴于:

对于 IA32 架构，TARGET 值为“ia32”；对于 Intel64 架构，则为“intel64”。PIN_ROOT 是 Pin 套件的根目录。

使用 PinCRT for Windows（Clang-cl 工具包）进行构建

引言：

Pin 是一个基于 LLVM 的现代 C++ 库，它兼容 C++11 标准。该库取代了原有的基于 stlport 的 C++ 库，后者仅支持 C++03 标准。

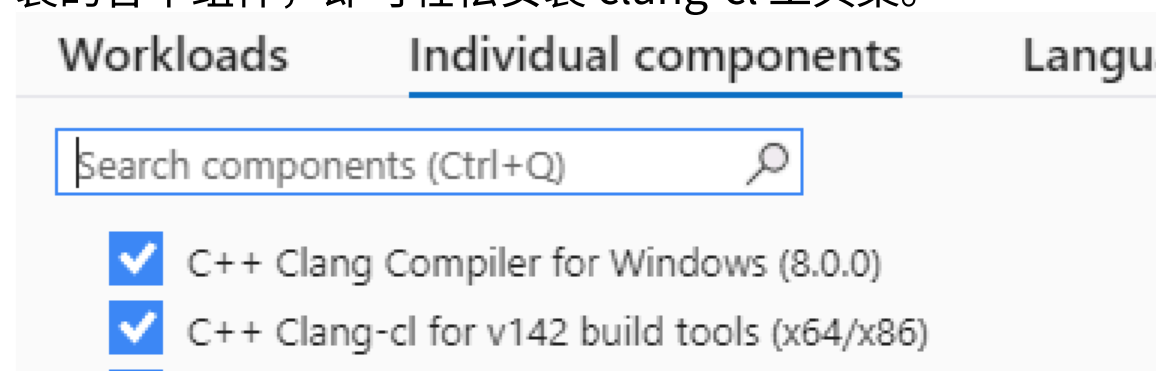
该库现在也可以在 Windows 操作系统上使用，它以独立的压缩包形式提供：pin-X.XX-buildnum-hash-clang-windows.zip。与基于 stlport 的旧版库相比，这两个库可以并行使用：pin-X.XX-buildnum-hash-msvc-windows.zip。

由于 LLVM 的局限性，要使用或编译这个新的 C++ 库，必须使用与 LLVM clang-cl 工具集兼容的 Pin Tools，而不能使用微软的工具集。不过，由于 Microsoft Visual Studio 支持在各种工具集之间切换，因此实际操作起来相当简单。以下部分将详细介绍如何使用 Pin Windows clang-cl 工具集。

先修要求：

使用 LLVM C++ 库来构建 Pin 工具时，最低要求是 Clang-cl 14.0.5 版本（建议使用 15.0.X 或更高版本）。

如果您已经安装了 Visual 2022，那么只需运行 Visual Studio 安装程序，然后选择要安装各个组件，即可轻松安装 clang-cl 工具集。



详情请参阅微软的官方文档。

安装完成后，可通过以下方式验证 clang-cl 的版本：

“C:\Program Files\Microsoft Visual Studio\2022\Professional\VC\Tools\Llvm\bin\clang-cl.exe”
-version

或者，也可以直接从 LLVM 的网站下载适用于 Windows 系统的 clang-cl。

使用 Visual Studio 进行构建

重要提示：以粗体标出的项目是 clang-cl 所特有的。如果您正在将代码从 MSVC 迁移至 clang-cl，请确保将这些项目添加到编译命令行中。

编译器：

请确保您的构建脚本/Make 文件使用的是 LLVM 的 clang-cl.exe，而非 Microsoft 的 cl.exe。

编译器参数：

宏指令：

- 适用于所有平台：
`/DPIN_CRT=1 /DTARGET_WINDOWS /D_LIBCPP_DISABLE_AVAILABILITY /D_LIBCPP_NO_VCRUNTIME /D__BIONIC__`
- 对于 IA32 架构：`/DTARGET_IA32 -m32 /D__i386__ /DHOST_IA32`
- 对于 Intel64 架构：`/DTARGET_IA32E -m64 /D__LP64__ /DHOST_IA32E`
- 用于构建 PinTool 或共享库：`/MD`
- 如需构建可执行文件或独立运行的工具：请使用 `/MD` 选项。

旗帜：

`/GR- /GS- /EHs- /EHa- /FP:严格模式 /Oi- -禁止为链接目的使用非类型定义的符号 -Wno-microsoft-include -Wno-unicode`

包含路径：

路径的格式应如下所示：

```
/I$(PIN_ROOT)/source/include/pin
/I$(PIN_ROOT)/source/include/pin/gen
/I$(PIN_ROOT)/extras/cxx/include
/I$(PIN_ROOT)/extras /I$(PIN_ROOT)/extras/crt/include
/I$(PIN_ROOT)/extras/crt
/I$(PIN_ROOT)/extras/crt/include/arch-$(BIONIC_ARCH)
/I$(PIN_ROOT)/extras/crt/include/kernel/uapi
/I$(PIN_ROOT)/extras/crt/include/kernel/uapi/asm-x86
/I$(PIN_ROOT)/extras/components/include
/I$(PIN_ROOT)/extras/xed-$(XED_ARCH)/include/xed
```

鉴于：

BIONIC_ARCH 在 IA32 架构下为“x86”格式，在 Intel64 架构下则为“x86_64”格式。XED_ARCH 在 IA32 架构下为“ia32”格式，在 Intel64 架构下则为“intel64”格式。PIN_ROOT 则是 Pin 套件的根目录。

注意：如果您正在将代码从 MSVC 迁移至 clang-cl，请务必删除以下内容。

来自包含路径：

```
/I$(PIN_ROOT)/extras/stlport/include  
/I$(PIN_ROOT)/extras/libstdc++/include
```

并将其替换为：

```
/I$(PIN_ROOT)/extras/cxx/include
```

附加的包含文件：

你应该将此开关添加到编译命令行中，这样在每次编译时都会包含该文件：

```
/FIinclude/msvc_compat.h
```

重要提示：

如果你在某个源文件中引用了 Windows.h：

我们提供了自己的替代文件来替换这个头文件（因为包含 Windows.h 的话，通常也会把所有不需要的 MS-CRT 相关内容一起包含进来）。我们所提供的替代文件实际上是一个“包装器”头文件，它能够确保只有我们需要的内容被声明出来。为此，我们必须知道原始的 Windows.h 文件位于何处——该文件位于 Windows SDK 中。在这种情况下，你需要将这个文件添加到编译参数中。

```
/D_WINDOWS_H_PATH_="$(ORIGINAL_WINDOWS_H_PATH)"
```

链接器参数：

链接器标志：

首先，应从链接器命令行中删除微软的 libc 库文件（例如：libcpmt.lib 或 libcmtd.lib）。这些库需要被 PinCRT 版本的库所替代。

将这些导入库添加到链接命令行中：

- 用于构建 PinTool 或共享库：
`/safeseh:no /NODEFAULTLIB pincrt.lib pinipc.lib c++.lib kernel32.lib`
- 如需构建可执行文件或独立运行的工具：
`/safeseh:no /NODEFAULTLIB pincrt.lib c++.lib kernel32.lib`

仅支持动态版本的 PinCRT 库。

如果你正在将代码从 MSVC 移植到 clang-cl：

1. 请确保将上面加粗标注的项添加到链接器的命令行中。
2. 将原本对 Microsoft Linker (link.exe) 的调用，替换为对 LLVM Linker (lld-link.exe) 的调用。

图书馆搜索路径：

```
/LIBPATH:${PIN_ROOT}/${TARGET}/runtime/pincrt  
/LIBPATH:${PIN_ROOT}/${TARGET}/lib
```

鉴于：

对于 IA32 架构，TARGET 值为“ia32”；对于 Intel64 架构，则为“intel64”。PIN_ROOT 是 Pin 套件的根目录。

忽略链接器警告：

可以忽略链接器警告#4210 和#4049。只需在链接器命令行中添加相应的参数，即可屏蔽这些警告：

```
/忽略:4210 /忽略:4049
```

需要关联的其他对象/项目

对于每个 libc 来说，都存在一个必须与所有可执行文件/共享对象静态链接的静态部分。具体需要链接哪些目标文件，取决于你是要构建可执行文件还是共享对象。

用于构建共享库或 PinTool：

- 该目标文件必须放在链接命令的首位：

```
crtbeginS.o
```

如需构建可执行文件或独立运行的工具：

- 该目标文件必须放在链接命令的首位：

```
crtbegin.o
```

上述目标文件位于以下位置：

```
${PIN_ROOT}/${TARGET}/runtime/pincrt
```

鉴于：

对于 IA32 架构，TARGET 值为“ia32”；对于 Intel64 架构，则为“intel64”。PIN_ROOT 是 Pin 套件的根目录。

使用 Pin 与 Clang-cl 时的问题——已知问题与解决方案

在 32 位工具中，对 Analysis 函数调用时，不要使用 fastcall 调用约定。

在 32 位工具中，当使用快速调用机制以及将大整数作为函数参数时，会存在严重的兼容性问题。在这些问题得到解决之前，请不要将快速调用机制与分析程序一起使用。该问题预计会在 clang 16 版本中得到修复。

Clang 编译器中的类型转换问题

使用“clang”进行类型转换并非默认支持的，必须在代码中明确声明。具体来说，clang 在函数调用时不会自动将(char *)类型转换为(void *)类型。这种情况下会引发编译错误。为了解决这个问题，需要手动将类型转换为(void *)。其他指针类型的转换也应遵循同样的规则。

Clang 优化功能

Clang 编译器拥有更复杂的优化功能。因此，在编写代码时，请务必先禁用该优化功能（使用/Od 选项）。等代码编写完成后，再切换到/O3 选项进行优化。如果优化过程中出现问题，很可能是由于 Clang 的优化机制导致的。

Std::tr1 已被::std 取代。

在 Stdport 中，一些 C++类（如 unordered_map、unordered_set 等）位于 std::tr1 命名空间下。现在，这些类被移至::std 命名空间下。请相应地修改您的代码。

将警告视为错误来处理。

默认情况下，Clang 在处理警告时的严格程度远远高于 Microsoft CL；Clang 会将这些警告视为错误。如果你的代码对这类警告比较敏感，可以事先使用-Wno-<>指令来禁用这些警告。

示例：

-Wno-未知的 pragma 指令 -Wno-指针符号问题 -Wno-不兼容的指针类型……

MMX 和 AVX 指令的处理方式（以及一般意义上的指令处理方式）

Clang 更倾向于遵循 GCC 的标志符约定，而非微软编译器的约定。具体来说，在使用 MMX/AVX 指令集时，那些在微软编译器中通过一个标志位来控制的功能，可能需要在 GCC 编译器中拆分成多个独立的标志位来控制。以下是一些例子：

```
/arch:AVX → -march=skylake-avx512
/arch:AVX → -mfma
/arch:AVX → -mxsavec -mavx
/arch:AVX → -maes -mpclmul -mssse3
```

Clang-Cl 优化对 mxcsr 寄存器的影响

Clang-cl 优化方式可能会影响 mxcsr 的功能，进而导致在使用内置函数时出现指令执行顺序混乱的问题。例如，如果在以下代码之后执行四舍五入操作，可能会导致计算结果出错：

```
__m128 ma = _mm_load_ss(&af);
__m128 mb = _mm_load_ss(&bf);
```

```
__m128 mres = _mm_mul_ss(ma,  
mb); _mm_store_ss(&rf, mres);
```

为了解决这个问题，请按照以下示例将相关寄存器声明为“易失性”寄存器：

```
volatile __m128 ma = _mm_load_ss(&af);  
volatile __m128 mb = _mm_load_ss(&bf);  
volatile __m128 mres = _mm_mul_ss(ma,  
mb);
```

Clang-cl 支持 GCC 的 `asm volatile` 约定。因此，另一种替代方案是自行定义符合 Linux 风格的汇编代码来实现这一功能。

```
__asm__ __volatile__ ("movss %1, xmm1 \n\t" // xmm1 = af  
    "movss %2, xmm2 \n\t" // 将 xmm2 的值复制到 xmm2 中  
    "mulss xmm2, xmm1 \n\t" // 将 xmm1 的值与 xmm2 的值相  
    乘  
    "movss xmm1, %0 \n\t" // 将结果存储到 rf 中  
    输出结果：“m” (af), “m” (bf)  
    输入参数：“%xmm1” , “%xmm2”
```

请注意，上述方法有助于您编写出更出色的交叉编译器代码。

从原生 CRT 格式迁移至 PinCRT 格式

消除与操作系统相关的函数调用

一般来说，PinCRT 仅支持 ISO C 标准中的函数。

总的来说，PinCRT 中没有实现任何与特定操作系统相关的功能（不过，对于我们选择的一些功能，还是实现了与特定操作系统相关的功能）。

请确保你使用了正确的头文件。

在使用 PinCRT 构建 Pin 工具时，所有的编译单元（即被编译成目标文件的源文件）都必须只包含下面列出的两类头文件。如果所包含的头文件不属于这两类中的任何一类，那么你的 Pin 工具很可能无法成功编译、链接或运行。PinCRT 支持的两类头文件如下：

1. PinCRT 和 Pin 头文件（这些文件位于 Pin 工具包的子目录中）。
2. 支持 PinCRT 功能的系统头文件——请查看支持 PinCRT 功能的系统头文件列表。

如何确保我的代码源中只包含与 PinCRT 兼容的头文件：

我们建议使用 GCC/Clang 的“-E”选项，或 Visual Studio 的“/E”选项，来获取 C/C++ 预处理器的原始输出结果。通过这些输出内容，您可以了解在源文件编译过程中被引用的所有文件。请确保所有被引用的文件（无论是直接被引用，还是通过其他方式间接被引用）都符合上述两类中的某一种。

支持 PinCRT 的系统头文件/代码片段

以下的头文件属于原生 CRT 组件，不过也可以通过支持 PinCRT 功能的 Pin 工具来包含这些头文件。

Linux（来自 glibc 的头文件）

- stddef.h
- stdarg.h
- float.h

Windows（Visual Studio 所使用的 libc 库）

- intrin.h
- xmmintrin.h
- windows.h（但有一些限制）。

请确保将您的 Pin 工具与 PinCRT 库正确关联起来。

当将 Pin 工具与 PinCRT 连接在一起时，该工具所依赖的所有库也必须与 PinCRT 一起被链接进来。

此外，也无法与现有的 C 运行时库进行链接（比如 glibc 或 Visual Studio CRT）。这意味着您可能需要重新编译工具所依赖的某些库，以便它们能与 PinCRT 正确链接。如果工具所依赖的某个库（无论是直接还是间接依赖的）与 PinCRT 不同的 C 运行时库相关联，那么在使用 PinCRT 运行该工具时就会导致程序崩溃。至于如何确定工具所依赖的库，需要具体查看相关文档。

在 Linux 系统上

在 Linux 终端中，运行以下命令：ldd <你的工具共享对象文件>。你应该能看到以下有效的 PinCRT 库：

- libc-dynamic.so
- libdl-dynamic.so
- libm-dynamic.so
- libc++.so
- libc++abi.so
- libpindwarf.so
- libdwarf.so
- libxed.so
- linux-vdso.so.1（虽然不属于 PinCRT 的组成部分，但仍然有效）。

以下是那些*不应该*出现在输出结果中的库。

请注意，如果您发现某个共享对象依赖于下面列出的某个库，

那么，这个共享对象必须使用 PinCRT 重新构建，这样才能确保它不依赖于其中的任何一个组件。
图书馆：

- libc.so.6
- libm.so.6
- libgcc_s.so.1
- libstdc++.so.6
- 位于/lib、/lib32 和/lib64 目录下的所有库文件

在“ldd”的输出结果中列出的其他库，也都需要再次使用“ldd”来检查其依赖的库。

在 Windows 系统上

在 Visual Studio 的命令行界面中，运行以下命令：

`dumpbin /DEPENDENTS <你的工具 DLL 文件>`。这样就能查看该 DLL 所依赖的所有组件。其中，那些列出的 PinCRT 库，正是你所需要的有效依赖库。

- `pincrt.dll`
- `kernel32.dll`——目前可以使用，但在 Pin 工具的未来版本中可能无法正常使用。原因是，使用该库时，Pin 工具可能会对其他应用程序造成不必要的干扰。
- 你在链接时明确指定的非系统 DLL 文件。你已经使用“`dumpbin`”工具对它们进行了检查。

如果在依赖项列表中还发现了其他库，那么我们必须使用 PinCRT 重新构建我们所分析的库，这样它就不再依赖那些其他库了。

使用 PinCRT 重新构建任何第三方库

如果你的 Pin 工具使用了第三方库，那么很可能需要使用 PinCRT 从源代码重新编译该库。实际上，只有当该第三方库完全不需要 `libc` 的支持时，才不必重新编译——不过这种情况相当罕见。

迁移过程中可能出现的常见问题

问题：

在尝试运行 Pin 时，您可能会遇到以下错误消息之一：

E: 无法加载 XXX: dlopen 函数调用失败: 无法找到“XXX”所引用的“stdout”符号……E: 无法加载 XXX: dlopen 函数调用失败: 无法找到“XXX”所引用的“stderr”符号……

解决方案：

你很可能是在使用原生 CRT 头文件来构建你的工具的（或者至少，是使用了与工具相关的那些目标文件中所包含的 CRT 头文件），而不是 PinCRT 头文件。

1. 找出那些因为使用了错误的头文件而被错误编译的文件。
2. 请确保在处理这些目标文件的编译命令行中，加入了所有必要的编译参数。
3. 如果您确定自己使用了正确的编译命令行，请参阅“如何确保我的源代码仅包含与 PinCRT 兼容的头文件”这一部分。

问题：

在尝试构建 Pin 工具时，您可能会遇到以下错误消息之一：

编译器错误：

错误：异常处理功能已禁用。如需启用该功能，请使用参数-fexceptions。

链接器错误：

对“cxa_allocate_exception”的引用无效……对“cxa_throw”的引用也无效。
在某个函数中引用了未解析的外部符号 CxxThrowException@8。

解决方案：

PinCRT 目前不支持 C++ 的异常处理机制。因此，您需要重新编写程序，避免使用异常处理。如果您遇到了链接错误，请确保已添加本文档中指定的所有 PinCRT 编译选项——具体来说，对于 Clang 和 GCC，需使用选项-fno-exceptions；对于 Visual Studio，则需使用选项/EHs 和/EHa-。这样，编译器会指出代码中所有使用 C++ 异常处理的地方。

在尝试构建 Pin 工具时，您可能会遇到以下错误消息之一：

编译器警告：

警告 C4541：在具有/GR-选项的情况下，对多态类型“XXX”使用了“dynamic_cast”操作；这可能导致不可预测的行为。

链接器错误：

错误 LNK2019：函数中引用的外部符号 RTDynamicCast 未找到。

...

错误 LNK2001：未解析的外部符号“const type_info::`vftable'”。同时，还出现了对“dynamic_cast”、“cxa_bad_cast”的引用错误；另外，对“cxxabiv1::vmi_class_type_info”和“cxxabiv1::class_type_info”的 vtable 的引用也出错了。此外，还出现了对“gxx_personality_v0”的引用错误。

解决方案：

PinCRT 目前不支持 C++ 的运行时类型信息处理功能以及动态类型转换。因此，您需要重新编写程序，避免使用这些功能。如果您遇到了链接错误，请确保已添加本文档中指定的所有 PinCRT 编译选项——具体来说，在 Clang 和 GCC 中需使用 -fno-rtti 选项，在 Visual Studio 中则需使用 /GR-选项。这样，编译器会指出代码中所有使用了 C++ 运行时类型信息处理功能或动态类型转换的部分。

问题：

在尝试构建 Pin 工具时，您可能会遇到以下错误消息之一：

链接器错误：

错误 LNK2019：未找到外部符号 @security_check_cookie@4 的定义。

在职能上……

错误 LNK2019：函数中引用的外部符号 security_cookie 未找到。

...

对“stack_chk_fail”的引用无效/无法找到“stack_chk_fail”这个函数/变量

解决方案：

您试图链接的一个或多个目标文件是用带有堆栈保护功能的编译器编译的（在 Visual Studio 中则称为安全检查功能）。请找出那些存在问题的目标文件——也就是那些被链接器报告为“符号引用未定义”的目标文件。同时，请确保您已使用了本文档中指定的所有必要编译选项：在 Clang 和 GCC 中请使用 -fno-stack-protector 选项，在 Visual Studio 中则请使用 /GS-选项。

在尝试构建 Pin 工具时，您可能会遇到以下错误消息之一：

链接器错误：

“dso_handle” 引用未定义。错误 LNK2001：外部符号 FLTUSED 未解析。错误 LNK2001：外部符号 atexit 未解析。错误 LNK2019：在函数 Ptrace_DllMainCRTStartup 中引用了未解析的外部符号 _CRT_INIT。

解决方案：

你可能没有将 Pin 工具与以下文件中的某个或几个进行关联：crtbeginS.o、crtbegin.o、crtendS.o、crtend.o、crtbeginS.obj、crtbegin.obj。具体需要关联哪些文件，取决于 Pin 工具所针对的操作系统，以及你是想构建共享库还是可执行文件。

详情请参阅具体的链接说明。

问题：

在将 Pin 工具从 MS-CRT 切换到 PinCRT 之后，所有的 *wprintf() 函数都开始出现异常行为。

特别是，“%s” 这种格式化方式无法正确地处理字符串。

解决方案：

MS-CRT 中的 *wprintf() 函数与其他几乎所有同类函数之间存在差异。

CRT 相关函数（包括 PinCRT）的 *wprintf() 函数：

- 在 MS-CRT 环境中：*wprintf() 函数中的默认字符串格式为宽字符（UTF16）。因此，在 *wprintf() 函数中使用 “%s” 实际上相当于使用 “%ls”。
- 在 PinCRT（以及 glibc 中）：*wprintf() 函数所使用的默认字符串都是多字节字符（即 UTF8 编码的字符）。

你从 MS-CRT 中移植过来的代码中，那些调用 *wprintf() 的函数，很可能期望在格式字符串中用 “%s” 表示的位置，实际传递的是宽字符。因此，最好的解决办法是将格式字符串中所有出现的 “%s” 都替换为 “%ls”。

问题：

在 Visual Studio 中，当尝试构建 Pin 工具时，可能会遇到以下错误消息之一：

链接器错误：

错误 LNK2019：外部符号 Cilog 未定义。错误 LNK2019：外部符号 libm_sse2_log_precise 未定义。或者类似以下的错误：错误 LNK2019：外部符号 Ci* 未定义。错误 LNK2019：外部符号 libm_* 未定义。

解决方案：
你在编译包含缺失引用的目标文件时，忘了添加/Oi-编译器标志。